

Web Service Design and Practice

06. Express

Dr. Kyudong Park

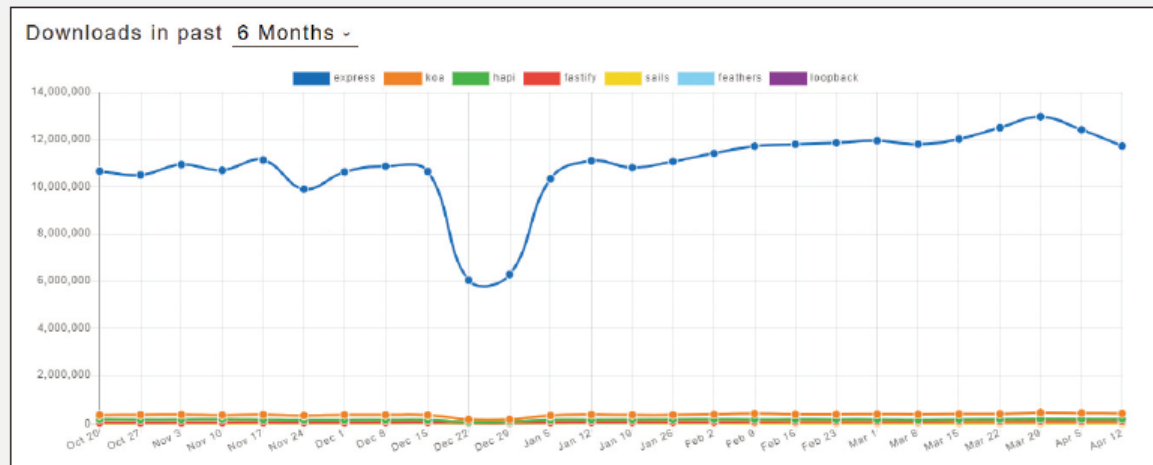
School of Information Convergence



1. Express 소개

- http 모듈로 웹 서버를 만들 때 코드가 보기 좋지 않고, 확장성도 떨어짐
 - 프레임워크로 해결
 - 대표적인 것이 Express(익스프레스), Koa(코아), Hapi(하피)
 - 코드 관리도 용이하고 편의성이 많이 높아짐

▼ 그림 6-1 express, koa, hapi 다운로드 수 비교



2. express 프로젝트 만들기

- npm init 명령어 생성
 - nodemon이 소스 코드 변경 시 서버를 재시작해줌

콘솔

```
$ npm i express  
$ npm i -D nodemon
```

package.json

```
{  
  "name": "learn-express",  
  "version": "0.0.1",  
  "description": "익스프레스를 배우자",  
  "main": "app.js",  
  "scripts": {  
    "start": "nodemon app"  
  },  
  "author": "ZeroCho",  
  "license": "MIT"  
}
```

2. express 프로젝트 만들기

- app.js 만들어보기
 - 가장 기초적인 node.js 기반 express 서버

app.js

```
const express = require('express');

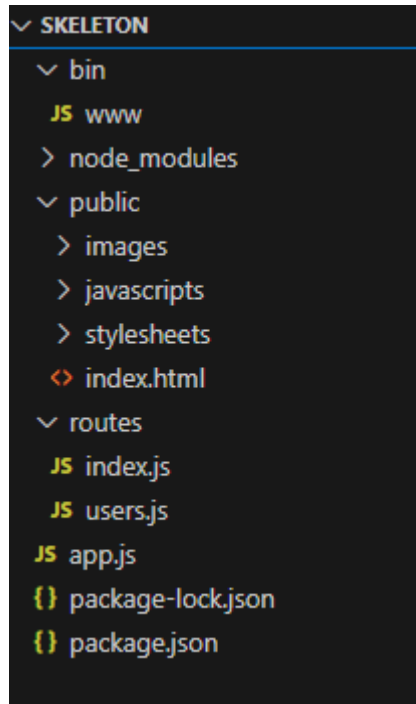
const app = express();
app.set('port', process.env.PORT || 3000);

app.get('/', (req, res) => {
  res.send('Hello, Express');
});

app.listen(app.get('port'), () => {
  console.log(app.get('port'), '번 포트에서 대기 중');
});
```

2. express 프로젝트 만들기

- `$ npm install -g express-generator`
 - global 환경에서 설치
 - generator
 - Use the application generator tool to quickly create an application skeleton.
- `$ express [PJT이름] --no-view`
 - no-view option : 뷰 템플릿 엔진을 사용하지 않음
- 자동으로 폴더구조와 기본 파일들을 생성해줌
- 최초 npm install 이후 사용 가능



2. express 프로젝트 만들기

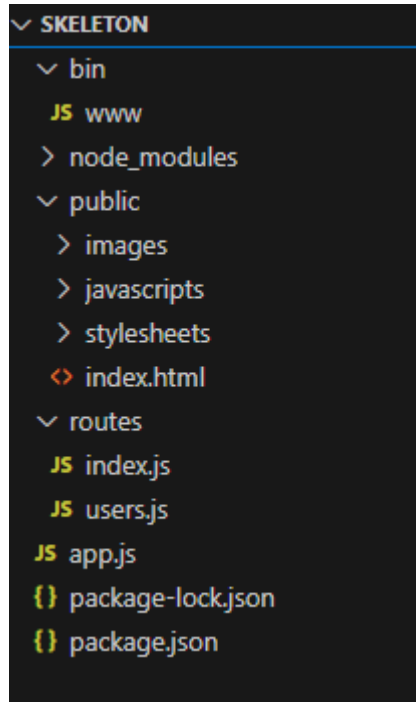
- jade (pug)
 - HTML 템플릿 엔진
 - 서버에서 HTML 을 만들어서 전송하는 방식 (서버와 클라이언트가 독립적이지 않음)
 - react 로 독립적인 프론트엔드를 구성하고 API 로만 통신하는 것이 주요 추세
 - 본 수업에서는 활용하지 않음
 - --no-view 옵션

Options:

```
--version      output the version number
-e, --ejs      add ejs engine support
--pug          add pug engine support
--hbs          add handlebars engine support
-H, --hogan    add hogan.js engine support
-v, --view <engine> add view <engine> support (dust|ejs|hbs|hjs|jade|pug|twig|vash) (defaults to jade)
--no-view      use static html instead of view engine
-c, --css <engine> add stylesheet <engine> support (less|stylus|compass|sass) (defaults to plain css)
-h, --help     output usage information
```

2. express 프로젝트 만들기

- bin/www: 서버를 실행하는 스크립트
- app.js: 핵심 서버 스크립트
- public: 외부에서 접근 가능한 파일들 모아둠
 - image, js, css 파일
- routes: 서버의 라우터와 로직을 모아둠
 - 추후에 models를 만들어 데이터베이스 사용



app.js 비교

- 서버 구동의 핵심이 되는 파일
 - app.set('port', 포트)로 서버가 실행될 포트 지정
 - app.get('주소', 라우터)로 GET 요청이 올 때 어떤 동작을 할지 지정
 - app.listen('포트', 콜백)으로 몇 번 포트에서 서버를 실행할지 지정

app.js

```
const express = require('express');
```

```
const app = express();
```

```
app.set('port', process.env.PORT || 3000);
```

```
app.get('/', (req, res) => {  
  res.send('Hello, Express');  
});
```

```
app.listen(app.get('port'), () => {  
  console.log(app.get('port'), '번 포트에서 대기 중');  
});
```

generator 에서는
bin/www 에 명시

generator 에서는
router 활용하여 명시

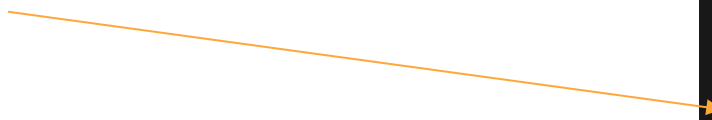
generator 를 활용하지 않은 case

3. view engine 설정

- \$ npm install ejs
- app.js 에 view engine 등록

```
app.engine("html", require("ejs").renderFile);  
app.set("view engine", "html");
```

- views 폴더 만들기
 - 하위에 serving 할 html 파일 생성
 - public 의 index.html 파일을 이동



```
✓ SKELETON  
  ✓ bin  
    JS www  
  > node_modules  
  ✓ public  
    > images  
    > javascripts  
  ✓ stylesheets  
    # style.css  
  ✓ routes  
    JS index.js  
    JS users.js  
  ✓ views  
    <> hello.html  
    <> index.html  
  JS app.js  
  {} package-lock.json  
  {} package.json
```

4. 라우터 객체

- app.js가 길어지는 것을 막을 수 있음
 - userRouter의 get은 /user와 /가 합쳐져서 GET /user/가 됨

routes/index.js

```
const express = require('express');

const router = express.Router();

// GET / 라우터
router.get('/', (req, res) => {
  res.send('Hello, Express');
});

module.exports = router;
```

routes/user.js

```
const express = require('express');

const router = express.Router();

// GET /user 라우터
router.get('/', (req, res) => {
  res.send('Hello, User');
});

module.exports = router;
```

app.js

```
...
const path = require('path');

dotenv.config();
const indexRouter = require('./routes');
const userRouter = require('./routes/user');
...
name: 'session-cookie',
}});

app.use('/', indexRouter);
app.use('/user', userRouter);

app.use((req, res, next) => {
  res.status(404).send('Not Found');
});

app.use((err, req, res, next) => {
  ...
```

4. 라우터 객체

- :id를 넣으면 req.params.id로 받을 수 있음
 - 동적으로 변하는 부분을 라우트 매개변수로 만들

```
router.get('/user/:id', function(req, res) {  
  console.log(req.params, req.query);  
});
```

- 일반 라우터보다 뒤에 위치해야 함

```
router.get('/user/:id', function(req, res) {  
  console.log('애만 실행됩니다.');
```

```
});  
router.get('/user/like', function(req, res) {  
  console.log('전혀 실행되지 않습니다.');
```

```
});
```

- /users/123?limit=5&skip=10 주소 요청인 경우

콘솔

```
{ id: '123' } { limit: '5', skip: '10' }
```

4. 라우터 객체

- 요청과 일치하는 라우터가 없는 경우를 대비해 404 라우터를 만들기
 - app.js

```
app.use((req, res, next) => {  
  res.status(404).send('Not Found');  
});
```

- 이게 없으면 단순히 Cannot GET 주소 라는 문자열이 표시됨

4. 라우터 객체

- 라우터 그룹화 하기
 - 주소는 같지만 메서드가 다른 코드가 있을 때

```
router.get('/abc', (req, res) => {  
  res.send('GET /abc');  
});  
  
router.post('/abc', (req, res) => {  
  res.send('POST /abc');  
});
```

- router.route로 묶음 (chaining)

```
router.route('/abc')  
  .get((req, res) => {  
    res.send('GET /abc');  
  })  
  .post((req, res) => {  
    res.send('POST /abc');  
  });
```

5. 익스프레스 서버 실행하기

- npm start(package.json의 start 스크립트) 콘솔에서 실행

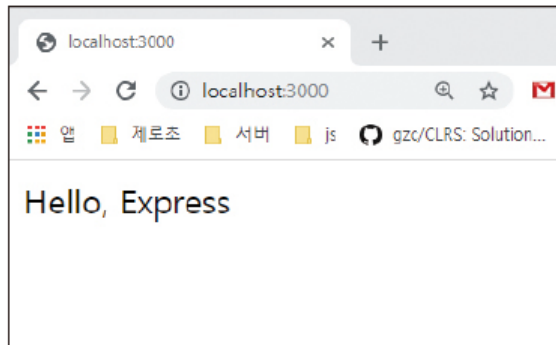
콘솔

```
$ npm start  
> learn-express@0.0.1 start C:\Users\zerocho\learn-express  
> nodemon app
```

[nodemon] 2.0.3

- localhost:3000

▼ 그림 6-2 localhost:3000 접속 화면



6. 배포

- Cloud 서비스에 node 설치
- Project code clone
 - 방법1. copy & paste
 - 방법2. git based (recommend)
- node 실행 without nodemon
 - nodemon 은 개발 환경에서의 편의를 위한 것 (실 배포 환경에서는 수행되어선 안됨)
 - 지속적으로 node 프로세스를 수행하도록 만들기
 - 방법1. nohup
 - 방법2. pm2 (recommend)

Thank you

Q & A